

RIDESYNC: AN INTELLIGENT CAMPUS MOBILITY, ROUTE MATCHING AND DRIVER DISPATCH PLATFORM

Comprehensive Technical & Architectural Project Report for Academic Evaluation

System Architecture | Geodesic Route Fitting | Pessimistic Locking | Security & Verification | Mobile Responsive Design System

1. Executive Summary & Abstract

RideSync is an end-to-end, high-performance web-based campus mobility and intelligent driver dispatch system. Engineered specifically to solve last-mile commute challenges across university ecosystems, RideSync provides a unified workspace for student riders, verified campus drivers, and system administrators. By integrating geodesic route-fit matching algorithms, real-time fallback dispatching, persistent session state management, and 1-tap emergency SOS notifications, RideSync guarantees campus safety, strict concurrency control, and operational efficiency.

Key Project Highlights & Achievements:

- *Multi-Role Mobile Responsive Architecture: Unified Rider, Driver, and Admin portals with sticky glassmorphic navigation, safe-area inset compliance, and responsive bottom dock.*
- *Algorithmic Route Matching: Haversine geodesic distance calculations, vector direction alignment, and time window fit scoring.*
- *High Concurrency & Lock Protection: Row-level SELECT ... FOR UPDATE database transaction locks preventing double-claim race conditions.*
- *Enhanced Security & Persistent Sessions: 30-day sliding session windows with instant middleware revocation for suspended user accounts.*
- *Automated AI Driver KYC Verification: Python FastAPI microservice utilizing OpenCV feature matching and document OCR for automated driver onboarding.*
- *Sub-50ms API Performance & 100% Quality Pass: OPcache TTFB optimizations, session principal caching, and 65 automated unit tests passing in PHPUnit with zero regression vulnerabilities.*

2. Technology Stack & System Architecture

RideSync is engineered using a decoupled, modular Model-View-Controller (MVC) architecture in PHP 8.2 and MySQL/MariaDB 8.4, complemented by a responsive Vanilla JavaScript frontend with centralized CSS design tokens, a Node.js WebSocket gateway for real-time telemetry, and an external Python AI KYC document verification service.

Layer / Component	Technology / Framework	Technical Purpose & Responsibility
Backend Web Core	PHP 8.2 (Strict Types, Mysqli, OPcache)	Request handling, auth middleware, session principal caching, CSRF token verification, action routing
Database Layer	MySQL 8.4 / MariaDB 10.4+ (Aiven SSL CA)	ACID relational persistence, spatial indexes, FOR UPDATE pessimistic

		locks, automated schema migrations
Frontend & Design System	Vanilla JS (ES6+), Glassmorphic CSS	Design tokens scale, responsive glassmorphic cards, notification toasts, live tracking, mobile bottom dock
Geolocation & Maps	OSRM & Leaflet / OpenStreetMap	Geodesic coordinate routing, pickup ETA calculation, route line polyline rendering
Realtime Telemetry	Node.js 20+ WebSocket Gateway	HMAC SHA-256 signed query tokens, low-latency SOS emergency alerts, driver live location broadcast
AI Driver Verification	Python 3.11/3.13 FastAPI + OpenCV	Automated driver KYC document OCR, facial feature matching, and background verification service
Cloud Infrastructure	Render Docker Blueprints & Vercel Edge	Vercel edge reverse proxying, Render Web Service containerization, S3 document storage abstraction
Quality & Testing	PHPUnit 11 & K6 Load Testing	65 automated unit tests, negative security tests, session security checks, sub-50ms performance load profiling

3. Core Mathematical Models & Algorithms

3.1 Geodesic Route Distance (Haversine Formula)

To compute precise spatial distances between pickup coordinates (lat1, lon1) and destination coordinates (lat2, lon2) on Earth, RideSync implements the spherical Haversine formula:

$$\begin{aligned} a &= \sin^2(\Delta lat / 2) + \cos(lat1) * \cos(lat2) * \sin^2(\Delta lon / 2) \\ c &= 2 * \operatorname{atan2}(\sqrt{a}, \sqrt{1 - a}) \\ \text{Distance } (d) &= R * c \quad (\text{where Earth Radius } R = 6,371 \text{ km}) \end{aligned}$$

3.2 Route Fit & Compatibility Scoring Equation

When a rider searches for rides, RideSync evaluates every candidate route using a multi-factor fitness function:

$$\text{Fit Score} = (W_{\text{dist}} * S_{\text{dist}}) + (W_{\text{time}} * S_{\text{time}}) + (W_{\text{seats}} * S_{\text{seats}})$$

Where:

- $S_dist = \max(0, 1 - (Pickup_Deviation_km / Max_Allowed_Radius))$
- $S_time = \max(0, 1 - (|Requested_Time - Offered_Time|_mins / 60))$
- $S_seats = (Available_Seats \geq Requested_Seats) ? 1.0 : 0.0$
- *Weights:* $W_dist = 0.50, W_time = 0.35, W_seats = 0.15$

3.3 Concurrency & Pessimistic Lock Prevention Algorithm

To eliminate race conditions when multiple drivers attempt to claim the same rider request simultaneously, RideSync enforces database-level transactions with SELECT ... FOR UPDATE locks:

1. *Start DB Transaction:* `mysqli_begin_transaction($conn);`
2. *Acquire Exclusive Lock:* `SELECT status FROM driver_requests WHERE id = ? FOR UPDATE;`
3. *Validate Status:* If status != 'pending', ROLLBACK transaction and return HTTP 409 conflict error.
4. *Mutate State:* `UPDATE driver_requests SET status = 'accepted', driver_id = ? WHERE id = ?;`
5. *Commit Transaction:* `mysqli_commit($conn);`

3.4 Persistent 30-Day Session & Instant Revocation Algorithm

RideSync implements a non-intrusive persistent session model that keeps authenticated users logged in for 30 days across browser restarts, combined with immediate security enforcement:

- *Session Cookie Lifetime:* 2,592,000 seconds (30 Days)
- *Garbage Collection Max Lifetime:* `ini_set('session.gc_maxlifetime', '2592000');`
- *Sliding Session Expiry:* Inactivity timeout refreshed on every request up to 30 days.
- *Instant Revocation Middleware:* `ridesync_validate_session_principal` checks user status in DB on every request; if status == 'suspended', session is terminated immediately and cookies destroyed.

4. Mobile Responsive Design System & UI Architecture

RideSync features a modern, mobile-first design system built using Vanilla CSS tokens, glassmorphic styling, and progressive enhancement without heavy external UI dependencies.

Design System Specifications:

- *Single-Row Sticky Glassmorphic Navbar:* Compact top bar housing logo mark, page header, unread alert bell badge, and user profile avatar dropdown.
- *Mobile Bottom Navigation Dock:* Fixed bottom tab bar providing 1-tap switching between Home, Search/Post, Driver Roster, Reports, and Settings.
- *Responsive Table-to-Card Transformations:* Complex tabular data automatically transforms into stacked glassmorphic cards on mobile viewports (<768px).
- *Ergonomic Touch Targets & Protection:* Minimum 44px x 44px interactive tap targets, safe-area-inset padding for notched devices, and disabled auto-zoom ('touch-action: manipulation').

5. Database Schema & Data Dictionary

The RideSync database contains 12 core relational tables configured with foreign key constraints, composite spatial/telemetry indexes, and full ACID compliance.

Table Name	Primary Keys / Indexes	Key Attributes	System Purpose
users	PRIMARY(id), UNIQUE(email)	name, email, password, status, role	Rider account credentials & instant suspension status
driver_accounts	PRIMARY(id), UNIQUE(email, phone)	name, phone, status, availability	Driver credentials, approval state & online availability status
driver_documents	PRIMARY(id), idx_driver_doc	driver_id, document_type, status	KYC documents (License, Aadhaar, PAN, RC) & AI review state
rides	PRIMARY(id), idx_status_date	user_id, origin, destination, seats	Posted shared trips and pickup/destination criteria
driver_requests	PRIMARY(id), idx_ride_driver	ride_id, driver_id, status, fare	Direct rider-to-driver dispatch requests with pessimistic locks
sos_alerts	PRIMARY(id), idx_status_id	ride_id, triggered_by_type, lat, lng	Active emergency SOS alerts & location coordinates
user_emergency_contacts	PRIMARY(id), idx_user_role	user_id, user_type, name, phone	Personal emergency contacts notified on SOS trigger
realtime_events	PRIMARY(id), UNIQUE(idempotency)	event_type, audience, payload_json	Idempotent event log for WebSocket push notifications

6. API Specifications & Action Endpoints

RideSync exposes modular action handlers and RESTful JSON endpoints for real-time interactions:

Endpoint / Action Route	Method	Parameters / Payload	Response / Effect
/api/v1/user_notifications_check.php	GET	since_id (int)	JSON unread count and latest alert toast payload
/actions/login_action.php	POST	email, password, csrf_token	Authenticates user, checks suspension status, sets 30-day session
/actions/match_action.php	POST	ride_id, action_type	Creates join request or updates match state with transaction locks

/actions/driver_request_action.php	POST	action_type, ride_id, fare	Executes FOR UPDATE locked driver request claim
/actions/emergency_contact_action.php	POST	action_type, name, phone	Adds or removes user emergency safety contacts
/actions/sos_action.php	POST	ride_id, latitude, longitude	Triggers SOS alert, broadcasts WebSocket event & notifies safety contacts

7. Security Architecture & Cloud Deployment

RideSync incorporates enterprise-grade security mechanisms across data persistence, session management, and deployment topology:

Security & DevOps Specifications:

- *Sensitive Document Encryption:* Driver KYC documents stored locally or in S3 are encrypted at rest using AES-256-CBC via `RIDESYNC_DOCUMENT_ENCRYPTION_KEY`.
- *CSRF & SQL Injection Protection:* All state-changing POST endpoints enforce anti-CSRF token verification; database interactions use prepared `mysqli` statements.
- *Content Security Policy (CSP):* Dynamic CSP header supporting local/LAN IP WebSocket connections (`'connect-src ws://127.0.0.1:8081'`).
- *Monorepo Cloud Topology:* Vercel Edge reverse proxy (`'vercel.json'`) routing traffic to Render Docker Web Service (`'render.yaml'`) connected to Managed Aiven MySQL via SSL CA certificate (`'ca.pem'`).

8. Quality Assurance, Testing & Performance Metrics

RideSync enforces automated quality gates prior to any production deployment:

- *PHPUnit Test Suite:* 65 automated unit tests covering authentication security, OTP validation, pessimistic locks, session security, and emergency contacts.
- *TTFB Latency Optimization:* OPcache bytecode caching, session principal caching, early session unlock, and Gzip compression delivering ultra-low page rendering latencies.
- *High Concurrency Stress Testing:* K6 load testing validating sub-50ms API response times under high concurrent user load.